

DAA - Assignment 1 (on chapter 1)

Name: ANUSKA NATH

Dept: IT year: UG2

Roll: 002311001003

Q1) Let A and B are two sorted lists such that A is in ascending order and B is in descending order. Write an algorithm to merge A and B such that the final list is in ascending order. Analyse and discuss the correctness of your algorithm.

Ans) Key idea: Given two sorted lists $\rightarrow$ A ascending, B descending.

- Traverse A from beginning.
- Traverse B from end (elements achieved in ascending order).
- Perform a standard two pointer merge.

Algorithm

merge-ASC (A, B):

$i \leftarrow 0$ // pointer for A

$j \leftarrow m-1$ // pointer for B

$k \leftarrow 0$

create array c of size $(n+m)$ // n is size of A, m is size of B

while $i < n$ and $j \geq 0$:

if $A[i] \leq B[j]$:

$c[k] \leftarrow A[i]$

$i \leftarrow i+1$

else:

$c[k] \leftarrow B[j]$

$j \leftarrow j-1$

$k \leftarrow k+1$

while $i < n$:

$c[k] \leftarrow A[i]$

$i \leftarrow i+1$

$k \leftarrow k+1$

while $j \geq 0$:

$c[k] \leftarrow B[j]$

$j \leftarrow j-1, k \leftarrow k+1$

return c

Example

$A = [1, 4, 6, 8]$

$B = [9, 7, 5, 2]$

result: $C = [1, 2, 4, 5, 6, 7, 8, 9]$

Algorithm steps:

- (1) Assign a pointer at start of A , and another at end of B .
- (2) Start a while loop that runs till both $i < n$ and $j \geq 0$ [both i and j are within the size of A and B]
- (3) check which element is smaller: one at index i of A or the one at index j of B .
- (4) enter the smaller value into C and increment value of i by 1 if element of A was smaller, otherwise ~~increment~~ decrement value of j by 1.
- (5) repeat step 3 again.
- (6) Once the loop breaks, enter the remaining elements of A or B (whichever was left) into C .
- (7) return array C .

Correctness

- 1) At the start of each iteration of the main while loop:
 - The array $C[0 \dots k-1]$ contains the k smallest elements from A and B in ascending order.
 - The next candidates are $A[i]$ and $B[j]$, the smallest remaining elements of their respective lists.
- 2) ~~Initialization~~ Initialization: Before the first iteration, $k=0$, so C is empty and initially sorted.
- 3) Maintenance:
 - At each step, the algorithm compares the smallest unprocessed elements of A and B .

- The smaller element is appended to C , maintaining sorted order.
- The corresponding pointer is advanced, preserving the invariant

4) Termination:

- When either list is exhausted, the remaining elements of the other list are already sorted and greater than all previously inserted elements.
- Appending them preserves ~~the~~ ascending order.

Conclusion

By induction, the final array C contains all elements of A and B in ascending order.

Hence, algorithm is correct.

Time complexity: $O(n+m)$

Space complexity: $O(n+m)$

Q2) Let A and B are two sorted lists such that A is in ascending order and B is in descending order. Write an algorithm to ~~merge~~ rearrange A and B such that all the elements ~~of~~ in A are less than or equal to all the elements in B . Analyse and discuss the correctness of the algorithm.

Ans) Key idea: Swap the elements in A that are larger than elements in B , with the element in B that is smaller.

Algorithm: (Two-Pointer Swap)

- 1) Initialise two pointer, i for A and j for B , both initialized to 0 (i is on smallest element of A , j is on largest element of B).
- 2) Start a loop that runs till both i and j are within size of A and B respectively.
- 3) Compare $A[i]$ and $B[j]$, if $A[i]$ is lesser than or equal to $B[j]$, skip to step 6.
- 4) If not, then swap $A[i]$ and $B[j]$.

5) increment both i and j by 1

6) sort A in ascending order and B in descending order.

Correctness

- Since A is ascending, $A[i]$ is the smallest unprocessed element in A.
- Since B is descending, $B[j]$ is the largest unprocessed element in B.
- If $A[i] > B[j]$, swapping places a larger element into B and a smaller one into A, moving the lists toward the desired condition.
- After all necessary swaps, sorting restores the order of both lists.
- Hence all elements of $A \leq$ all elements of B.

Complexity analysis

- Comparisons and swaps: $O(n+m)$
- Sorting A and B: $O(n \log n + m \log m)$

Overall: Time complexity: $O(n \log n + m \log m)$.

Q3) Let A be a list of unordered elements. Write an efficient algorithm to find min and maximum together together with no. of comparison $3n/2 - 2$.
Analyze and discuss the correctness of your algorithm.

Ans) Key idea: Compare elements in pair.

- First compare the two elements within a pair.
- Then compare the smaller one with current minimum and larger one with the current maximum.

Algorithm

1) Find out if n (size of A) is even or odd.

2) if n is odd: $\min \leftarrow A[0]$, $\max \leftarrow A[n]$.
 if n is even: $\min \leftarrow \min(A[0], A[1])$
 $\max \leftarrow \max(A[0], A[1])$

3) Start a for loop with i whose value starts at 2 or 1 for even or odd n respectively and goes upto $n-1$ with a step 2.

4) compare values of $A[i]$ and $A[i+1]$.
 If $A[i]$ is smaller, compare $A[i]$ with $\min$ and $A[i+1]$ with $\max$ and modify the values of $\min$ and $\max$ if the values fit better.
 otherwise, compare $A[i+1]$ with $\min$ and $A[i]$ with $\max$ and modify the values of $\min$ and $\max$ if values fit better.

5) return $\min$ and $\max$.

for odd n

$\min \leftarrow A[0]$

$\max \leftarrow A[0]$

for $i \leftarrow 1$ to $n-1$ step 2:

if $A[i] < A[i+1]$:

if $A[i] < \min$: $\min \leftarrow A[i]$

if $A[i+1] > \max$: $\max \leftarrow A[i+1]$

else:

if $A[i+1] < \min$: $\min \leftarrow A[i+1]$

if $A[i] > \max$: $\max \leftarrow A[i]$

for even n

$\min \leftarrow \min(A[0], A[1])$

$\max \leftarrow \max(A[0], A[1])$

for $i \leftarrow 2$ to $n-1$ step 2:

if $A[i] < A[i+1]$:

if $A[i] < \min$: $\min \leftarrow A[i]$

if $A[i+1] > \max$: $\max \leftarrow A[i+1]$

else:

if $A[i+1] < \min$:
 $\min \leftarrow A[i+1]$

if $A[i] > \max$:
 $\max \leftarrow A[i]$

Number of comparisons:

initial comparison: 1

number of pairs: $\frac{n}{2} - 1$

comparisons per pair: 3

Total comparisons = $1 + 3 \left(\frac{n}{2} - 1 \right)$

= $\frac{3n}{2} - 2$.

Time complexity: $O(n)$

Space complexity: $O(1)$.

Correctness

1) Loop Invariant:

At the start of each iteration of the while loop,

- min is the minimum of all elements processed so far.
- max is the maximum of all elements processed so far.

2) Initialization:

- Before the loop begins, min and max are correctly set using either the first element (odd case) or the first pair (even case).
- Hence loop invariant holds initially.

3) Maintenance:

- Each iteration processes two new elements.
- One comparison identifies their local minimum and maximum.
- The local minimum is compared only with min and the local maximum only with max.
- Thus, after updating, min and max remain correct for all processed elements.

4) Termination:

- The loop terminates when all elements have been processed.
- By ~~the~~ the loop invariant, min and max are the minimum and maximum of the entire list.

Conclusion

The algorithm correctly compares and computes the minimum and maximum of an unordered list using exactly $\frac{3n}{2} - 2$ comparisons, which is optimal.

Q4) Write a stable and Inplace sorting algorithm.
Analyse and discuss the correctness of your algorithm.

Ans) Algorithm:

- 1) for $i \leftarrow 2$ to n :
- 2) $key \leftarrow A[i]$
 $j \leftarrow i-1$
- 3) while $j \geq 1$ and $A[j] > key$:
 $A[j+1] \leftarrow A[j]$
 $j \leftarrow j-1$
- 4) $A[j+1] \leftarrow key$.

Time complexity: Best case: $O(n)$ Worst case: $O(n^2)$
Avg case: $O(n^2)$

Space complexity: $O(1)$

Correctness:

1) Loop invariant:

At the start of each iteration of the for loop, the subarray $A[1 \dots i-1]$ is sorted in ascending order and contains the same $\rightarrow$ elements as originally.

2) Initialisation:

- Before the first iteration ($i=2$), the subarray $A[1]$ has one element.
- A single element is ~~not~~ initially sorted.
- Hence, loop invariant holds.

3) Maintenance:

- The algorithm inserts $A[i]$ into its correct position within the sorted subarray $A[1 \dots i-1]$.
- Elements greater than key are shifted right, preserving order.
- Thus, after insertion, $A[1 \dots i]$ is sorted.

4) Termination

- The loop terminates when $i = n-1$
- By the loop invariant, the entire $A[1 \dots n]$ is sorted.

Conclusion

Insertion sort is a stable, in-place sorting algorithm that correctly sorts the input array.

Its correctness follows from the loop invariant, and it uses constant extra space.

Q5) Write an algorithm to find the 2nd minimum/maximum element from an unsorted list. Analyse ~~the~~ and discuss the correctness of your algorithm.

Ans) Algorithm (for 2nd minimum and 2nd maximum)

1) Initialise a list $L1[i]$ for each element $A[i]$ to store the element it defeats. ~~(do not for both min and max)~~

2) While more than one element remains:

a) Compare elements pairwise

b) The smaller element advances to next round for 2nd minimum and the larger element advances for 2nd maximum.

c) Store the other element in the winners list.

3) Let $\min_n$ and $\max_n$ be the final winners

4) Return the minimum element from $L1(\min)$ as second minimum and maximum element from $L2(\max)$ for second maximum.

Time Complexity:

- Finding the minimum or maximum requires $n-1$ comparisons.
- The minimum or maximum participates in $\lceil \log n \rceil$ comparisons.
- Finding the second minimum or maximum from these elements takes $\lceil \log n \rceil - 1$ comparisons.

$$T(n) = n + \lceil \log n \rceil - 2$$

Time complexity = $O(n)$

Space complexity = $O(n)$

Correctness

- 1) Loop invariant: At the start of each round
 - Each advancing element is the minimum or maximum among all elements it has been compared with.
 - For each advancing element, its list stores exactly the elements it has defeated.
 - 2) Initialization
 - Initially, no comparisons are made.
 - Each element is initially the minimum and maximum of itself.
 - Hence, the invariant holds.
 - 3) Maintenance
 - When two elements are compared, ~~the~~ for minimum the smaller advances and larger is stored, and for maximum, smaller is stored and larger advances.
 - The advancing element remains the minimum or maximum among all it has encountered.
 - Therefore, the invariant is preserved.
 - 4) Termination
 - When the tournament ends, the final winners ^{are} the minimum and maximum of the entire array.
 - The 2nd minimum and 2nd maximum must be the smallest and largest among elements that lost directly to the minimum and maximum respectively.
 - Selecting the minimum from min winner's list and maximum from the max winner's list yields correct 2nd min and 2nd max respectively.
- Conclusion: The algorithm is correct and uses optimal no. of comparisons.